Running head: Agentic ODE Parameter Estimation

An Agentic AI Workflow for Parameter Estimation of Complex Differential Equation Systems

Saakaar Bhatnagar[1]

[1] *<Affiliation, City, State/Country>*
ORCID: <ORCID ID>

Corresponding author: Saakaar Bhatnagar, <email address>

Abstract

Parameter estimation for ordinary differential equation (ODE) models fitted to experimental data is an iterative process often made difficult by heuristic fitting methods that make severely limiting assumptions, noisy experimental data, large search spaces and stiff ODE dynamics. Posing the estimation problem as an optimization problem is a commonly used method, but usually needs advanced knowledge of optimization algorithms and is prone to implementation errors and inefficiencies. This chapter describes an AI agent centered workflow for converting a lightweight estimation problem specification into a reproducible calibration pipeline. The overall task of estimation is broken down into sub steps, with the human in the loop confirming the agent's work. The agent's context for each step is populated with sub task-specific knowledge, enabling expert execution instead of random guesswork. The user provides key definitions in a structured file system. The workflow emphasizes reproducibility and audit through inspection of generated code, and diagnostic checks that help distinguish numerical, modeling, and data-quality failures. Overall, this workflow allows easy and robust iteration on model form, search space and objective functions, which critically affects the obtained models.

Key words

Parameter Estimation; Ordinary Differential Equations; Global Optimization; Differentiable Programming; Gradient-Based Optimization

1 Introduction

1.1 Motivation

add shaowu and nithin cites

Ordinary differential equation (ODE) models are widely used to represent biological, chemical, and physical systems, these often are put into bigger models, eg battery

A common strategy is to frame the fitting problem as an optimization problem, by minimizing the solution of the ODE or some derivative of it to the experimental data. include citations. Usually, practitioners employ global search methods, where the search is driven by population methods such as swarm optimization, differential evolution or genetic algorithms. These methods are agnostic to the physics of the problem, are trivially parallelizable, and are robust to sections of the search space being infeasible.

These methods return a good initial guess to the problem, which can be fine-tuned using gradient based methods to obtain a more accurate fit. An additional challenge for gradient based methods is that for numerical derivatives of the solution w.r.t parameters may be highly unstable or inaccurate. This depends on the numerical properties of the ODE and the dimensionality of the estimation problem. Recent work has shown that derivatives obtained via automatic differentiation of the solver are more stable and can be used to get accurate derivatives of solutions w.r.t parameters, which can be used for parameter updates, similar to the methods of adjoints refine and cite [1, 2]. Recent work has focused on including differentiable formulations of ODE systems directly in optimization, control, and reinforcement learning workflows [3, 4], motivating the need to work with differentiable formulations of dynamical systems.

A typical calibration exercise requires the user to translate equations into code, connect the code to observation data, define parameter bounds and transformations, select suitable numerical solvers, choose estimation algorithms, and diagnose failures when the fit is poor.

Executing all of the above steps requires expertise and effort to setup the entire estimation process. Any of these steps, if done incorrectly, can cause failure of the estimation. Some common failure modes include inappropriate scaling of parameters and objectives, an incorrect optimization setup, insufficient estimation algorithms, poor objective function construction, insufficient constraint flexibility, too broad or too narrow search ranges , and inefficient code implementations.

1.2 Agentic AI workflows

AI agents have recently been adopted across scientific computing for tasks such as geometry and CAD creation [5, 6], solver orchestration [7, 8], end-to-end simulation pipelines and post-processing [9, 10]. These workflows share a characteristic profile: they require substantial manual setup, proceed largely by trial and error, expose a large number of configuration knobs that materially affect the solution yet are easily overlooked, and demand expert knowledge to operate the underlying tools effectively. Crucially, their outcomes are often *verifiable*- the task's output can be checked against clear evidence, rules, tests, or ground truth rather than judged only by subjective preference. This combination of high manual overhead, expertise requirements, and built-in verifiability makes such workflows particularly amenable to augmentation by AI agents: the agent can shoulder the routine and knowledge-intensive setup. The success of these efforts, together with the recurring need for manual intervention and domain expertise, directly motivates applying AI agents to the ODE parameter estimation problem considered here.

However, delegating complex problems to an AI agent without user oversight risks hallucination, context degradation, and silent, incorrect assumptions, any of which can yield erroneous solutions [9, 11]. An agent left to run unsupervised through a long chain of decisions can therefore accumulate and amplify errors that a domain expert would catch immediately. A potential solution to these problems is using models with large parameter counts, large context windows, and extended reasoning capabilities. Models provided by major AI providers through cloud-based

services can be used to solve such problems via effective tool calling and the ability to directly process a large number of tokens. However, this methodology comes with issues around reproducibility and auditability, where the agent may make silent assumptions on different runs of the same problem and not surface them. More importantly, using cloud-hosted models to solve problems in scientific computing comes with privacy and intellectual property concerns, and often, practitioners insist on their data, mathematical models and workflow knowledge not leaving air-gapped computing environments that are locally managed.

The workflow described in this chapter is designed around these constraints, and describes a method to orchestrate local coding models to solve ODE parameter estimation problems. The broader fitting problem is decomposed into a sequence of sub-steps, each completed by the agent and producing intermediate artifacts written to disk. The decomposition enables effective utilization of context windows, and the tool performs a series of deterministic and LLM-based evaluations at every step. The tool identifies inconsistencies in the problem specification, repairs common input pathologies, transforms user-defined model functions into JIT-compilable (just-in-time compiled) routines, and assembles a two-stage optimization pipeline that combines a parallelized global search with gradient-based refinement. The models used in this workflow need not exceed approximately 40 billion parameters, making local deployment feasible on suitably equipped workstations

This chapter links two repositories: NAME1 which orchestrates the workflow as a set of skill files (utilizing remote LLMs), and NAME2 which runs an LLM locally. The workflow described in the chapter is focused on NAME2 since it has more limitations

2 Methods

2.1 Context Assisted Agents via division of tasks

A transformer language model defines an autoregressive distribution $p(x_t \mid x_{<t})$ over the next token x_t given all preceding tokens in its context window, repeatedly predicting the next piece of text from everything written so far. This conditioning is implemented by a mechanism called self-attention: as the model processes each token, it scores how relevant every earlier token is to the one currently being processed, using this relevance score to predict the next token. All task-relevant behavior is therefore induced at inference time by the context: the instructions, reference material, and examples assembled into the input are the primary levers for controlling an agent.

The context window is also a finite, shared resource, and attention makes its use non-uniform. For each token, the relevance scores described above are normalized so that they sum to one across all the earlier tokens (an operation called a softmax). Adding more tokens to the context window dilutes the effect that important tokens have by forcing them to share this fixed budget with a larger pool of competitors. Loading an entire domain into one static prompt thus scales poorly: it inflates cost and latency, and tends to lower quality as the signal-to-noise ratio of the window falls.

Hence, dividing a large task into sub-tasks, that determine which material is loaded into the context window precisely when that step executes, is effective: the rules the step must follow, the structural templates its output must match, and tool calls the LLM may make. Because the model attends over the sub-task, detail belonging to other steps is kept out of the window and out of the attention budget. This also keeps the active window dense with relevant tokens and well within

its length limit. This both sharpens output quality and lets each step be revised independently by editing its context alone.

2.2 Search Algorithms

2.2.1 Population methods

The framework ships with two population-based global optimizers, Particle Swarm Optimization (PSO) and Differential Evolution (DE), for the coarse search that precedes gradient-based refinement. Both are *gradient-free*, tolerating the stiff, non-smooth losses typical of ODE fitting; *global*, exploring an entire population rather than a single iterate to avoid poor local minima; and *embarrassingly parallel*, since per-candidate evaluations are independent and dispatched in parallel. The population is given a space-filling initialization via an optimized Latin Hypercube (LHS) design [12], and both search in the normalized $[-1,1]^d$ space so parameters spanning many orders of magnitude are treated on equal footing. The best candidate found seeds the gradient stage.

Particle Swarm Optimization (PSO) The global search defaults to PSO[13], a population-based metaheuristic in which a swarm of N candidate solutions (*particles*) explores the normalized parameter space $[-1,1]^d$. Particle i carries a position $\mathbf{x}_i \in [-1,1]^d$ and a velocity $\mathbf{v}_i$. The swarm is seeded with a LHS design to spread the initial positions evenly, and a star topology is used so every particle shares a single global best.

Let $f(\cdot)$ denote the (normalized) loss. Each particle tracks its own best position $\mathbf{p}_i$ and the swarm's global best $\mathbf{g}$:

$$\mathbf{p}_i \leftarrow \arg\min_{\mathbf{x}\in\{\mathbf{p}_i,\mathbf{x}_i\}} f(\mathbf{x}), \qquad \mathbf{g} \leftarrow \arg\min_i f(\mathbf{p}_i). \tag{1}$$

At each iteration k the velocity and position are updated by

$$\mathbf{v}_i^{k+1} = w\,\mathbf{v}_i^k + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^k) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^k), \tag{2}$$

$$\mathbf{x}_i^{k+1} = \mathbf{x}_i^k + \mathbf{v}_i^{k+1}, \tag{3}$$

where $\mathbf{r}_1, \mathbf{r}_2 \sim \mathscr{U}(0,1)^d$ are drawn element-wise, $\odot$ is the Hadamard product, w is the inertia weight, and c_1, c_2 are the cognitive and social coefficients. The velocity is clamped componentwise to $[-0.3, 0.3]$, and particles leaving the box are projected back to the nearest boundary.

Relative to differential evolution, PSO tends to converge faster on easier problem landscapes: the velocity memory and shared global attractor let information propagate through the swarm quickly, so good regions are exploited in fewer loss evaluations.

Differential Evolution (DE) As an alternative global search, the framework provides Differential Evolution [14], a population-based evolutionary algorithm well suited to multimodal, non-differentiable objectives. A population of N candidate vectors $\{\mathbf{x}_i\}_{i=1}^N$ in $[-1,1]^d$ is initialized with LHS and evolved by mutation, crossover, and selection.

For each target vector $\mathbf{x}_i$ at generation k, a *mutant* is formed from the current best and two distinct randomly chosen members ($r_1 \neq r_2 \neq i$):

$$\mathbf{m}_i = \mathbf{x}_{\text{best}}^k + F\left(\mathbf{x}_{r_1}^k - \mathbf{x}_{r_2}^k\right), \tag{4}$$

where the differential weight F is sampled per generation from $\mathscr{U}(0.5, 1.0)$ to improve robustness. A trial vector $\mathbf{u}_i$ is built by binomial crossover, mixing the mutant with the target at a per-component rate set by the recombination constant $CR = 0.7$:

$$u_{i,j} = \begin{cases} m_{i,j}, & \text{if } \rho_j \le CR \text{ or } j = j_{\text{rand}}, \\ x^k_{i,j}, & \text{otherwise,} \end{cases} \qquad \rho_j \sim \mathscr{U}(0,1), \tag{5}$$

where j_{rand} guarantees at least one component is taken from the mutant. Greedy selection retains whichever of the trial or target has lower loss:

$$\mathbf{x}^{k+1}_i = \begin{cases} \mathbf{u}_i, & \text{if } f(\mathbf{u}_i) \le f(\mathbf{x}^k_i), \\ \mathbf{x}^k_i, & \text{otherwise.} \end{cases} \tag{6}$$

Relative to PSO, DE is more robust on highly multimodal or ill-conditioned landscapes[14]: its difference-vector mutation adapts the search scale to the population's current spread, and the greedy, elitist selection prevents the population from drifting away from good solutions. It is therefore the safer choice when PSO stalls in a poor local minimum, at the cost of more loss evaluations to converge.

2.2.2 Gradient-based methods

The best candidate from the global search is refined by a gradient-based optimizer that minimizes the same normalized loss $f(\mathbf{x})$ over $[-1,1]^d$. Gradients are obtained by *automatic differentiation* (AD) through the full discretized ODE solve, rather than by finite differences. Because AD differentiates the discretized solver–objective map itself, it yields exact discrete sensitivities and requires no perturbation step size—a quantity that is notoriously hard to choose for stiff or highly nonlinear systems [15, 16]. In reverse mode, this is equivalent to a discrete adjoint approach, propagating sensitivities backward through a discretized solver to compute the loss gradient. The update is a projected step that keeps iterates inside the search box,

$$\mathbf{x}^{k+1} = \Pi_{[-1,1]^d}\big(\mathbf{x}^k + \Delta\mathbf{x}^k\big), \qquad \Delta\mathbf{x}^k = \mathscr{U}\left(\nabla f(\mathbf{x}^k), \mathbf{x}^k\right), \tag{7}$$

where Π is clipping onto the bounds and $\mathscr{U}$ is the optimizer's update rule. The framework exposes both a first-order method (Adam, with an exponentially decaying learning-rate schedule) and a second-order quasi-Newton method (L-BFGS); the best iterate is tracked throughout and returned, and a failed integration is caught so the search exits gracefully with the best solution so far.

This stage is intended for *fine-tuning* rather than global discovery: it converges quickly but needs a good initial guess, which the population-based search provides. It is resource-efficient and scales well with dimension, since AD computes the full gradient at a cost comparable to a single loss evaluation regardless of the number of parameters—unlike finite differences, whose cost grows linearly with dimension.

The gradient based search is also the more fragile of the two methods, and several failure modes exist. The computed gradient inherits the accuracy of the forward solve, so loosely converged or under-resolved integration injects noise into ∇f and can cause the iterates to diverge; the solver tolerances must generally be tighter for the gradient stage than for the global search to allow for a stable adjoint computation. Excessively noisy data has a similar effect, producing a rugged loss

surface whose local gradients point away from the true optimum. Some systems are ill-suited to differentiation: non-smooth dynamics (event triggers, clipping, piecewise or discontinuous right-hand sides) break differentiability. Even for smooth, well-behaved systems, convergence can stall on ill-conditioned problems where the loss has long flat valleys (near-degenerate parameter combinations), and a poor initial guess can leave the optimizer in the wrong basin since the local search cannot escape it. Finally, stiff systems may drive the adaptive solver to exceed its step budget, causing an integration failure mid-search; the framework guards against this by tracking the best iterate and exiting gracefully when a solve fails.

2.3 Post-Fit Sloppiness Diagnostic

After the gradient stage converges, the framework automatically characterizes how well the data constrain the fit by computing the local curvature of the loss at the optimum—the *sloppiness* spectrum using the methods of Gutenkunst *et al.* and Hass *et al.* [17, 18]. Let $\mathbf{u}$ denote the per-parameter axis with $u_i = \log_{10}\theta_i$ for parameters flagged `LOGSCALE` and $u_i = \theta_i$ otherwise, so curvature in a log-scaled parameter is measured per decade. Writing the converged best fit as $\mathbf{u}^\star$, the framework forms the Hessian of the normalized loss,

$$H \;=\; \nabla^2_{\mathbf{u}} f(\mathbf{u}^\star) \;\in\; \mathbb{R}^{d\times d}, \tag{8}$$

by second-order automatic differentiation through the entire ODE solve, falling back to a symmetric finite difference of the (first-order AD) gradient when higher-order differentiation through the stiff implicit solver is unavailable. The Hessian is symmetrized, $H \leftarrow \frac{1}{2}(H + H^\top)$, and diagonalized,

$$H \;=\; V\Lambda V^\top, \qquad \Lambda = \mathrm{diag}(\lambda_1 \geq \cdots \geq \lambda_d), \tag{9}$$

yielding an eigenvalue spectrum $\{\lambda_k\}$ and orthonormal eigenvectors $\{\mathbf{v}_k\}$. Each eigenvector is a *direction* in parameter space (a linear combination of the u_i) and its eigenvalue is the curvature of the loss along that direction: large λ_k marks a *stiff*, well-constrained combination, small λ_k a *sloppy*, poorly-constrained one, and $\lambda_k \to 0$ a flat, practically non-identifiable direction.

The reported verdict uses the eigenvalues normalized by the largest, $\hat{\lambda}_k = \lambda_k/\lambda_1$. A direction is flagged non-identifiable when $\hat{\lambda}_k$ falls below a floor $\varepsilon_0 = 10^{-6}$, and the fit is termed *sloppy* when the identifiable modes span more than six orders of magnitude,

$$\log_{10}\!\left(\frac{\lambda_1}{\lambda_{\min}^{\text{ident}}}\right) > 6 \qquad \text{or} \qquad \exists k : \hat{\lambda}_k < \varepsilon_0, \tag{10}$$

where $\lambda_{\min}^{\text{ident}}$ is the smallest eigenvalue above the floor.

This is a local, quadratic measure valid only at a converged optimum, so it detects the flat valleys that stall gradient descent. It is a measure of the quality of the current fit, but not whether there may be a better global minimum at some other solution. It should also be noted that this analysis does not instruct a user on which specific parameters are sloppy. However, if multiple runs under identical estimation conditions produce vastly different values of some parameters, that can be an empirical way to estimate the parameters that contribute most to the sloppiness spectrum.

3 Procedure

This section describes how to configure and run an ODE parameter-estimation problem using the local pfit workflow. The supported starting point is a reviewed mathematical problem description in `inputs/user_info.txt`, together with one or more CSV data files. The user is responsible for specifying the equations, parameter bounds, initial conditions, mappings from ODE system to observables, and mathematical objective. The framework then uses a language model to convert this specification into executable model code.

The workflow is divided into five command-line stages:

`pfit new` → `pfit check` → `pfit jax` → `pfit run` → `pfit diagnose`

During `pfit new`, the configured model extracts a structured fitting problem from `user_info.txt` and the declared CSV files. Deterministic then control whether the newly created `inputs/user_input.yaml` and `generated/user_model.py` are accepted. The `pfit check` stage examines the resulting data, configuration, model declarations, and objective, and writes a report without modifying the user-provided model.

The `pfit jax` stage uses the language model to translate the accepted Python model into JAX. The candidate translation is subjected to generated-code interface checks, numerical smoke tests, and sampled source-to-JAX comparisons. The `pfit run` stage repeats deterministic readiness checks and performs population-based exploration followed by gradient-based refinement. Each invocation creates a new preserved run directory, and post-fit sloppiness analysis is performed by default.

Finally, `pfit diagnose` reconstructs a completed run from its saved snapshot, calculates residual and optimization diagnostics, and generates measured-versus-simulated and residual plots. It also summarizes the sloppiness results produced during fitting. The configured model may be used to interpret the computed evidence, but it does not modify the session.

The separation between stages is deliberate. Each stage produces files that can be inspected before the next stage begins, making assumptions and generated code visible to the user. It also enables more efficient and accurate results from the language model, by only including necessary information in context for any step.

The following subsections first define the software, data, and problem-specification requirements and then describe each command and its generated artifacts in detail.

3.1 Setting up a problem

3.1.1 Initial software setup

The local workflow requires two components: a Python environment for running `pfit`, and a locally served model exposed through Ollama cite?. The Python package definition is stored in `pyproject.toml`, while the tested dependency set is pinned in `requirements-lock.txt`. For reproducible use, create a fresh virtual environment and install the locked environment:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements-lock.txt
```

It is **highly** recommended to use the software dependencies listed in the lock file, to avoid performance regressions. This installs the `pfit` command-line entry point together with the numerical and model-generation dependencies used by the workflow, including JAX, Diffrax, Optax, and the Ollama Python client. After installing Ollama, start the Ollama service and pull the selected coding model, for example:

```
ollama pull qwen3:30b-a3b
```

The model used by `pfit` can be configured in `pfit.yaml`:

```
llm:
  model: qwen3:30b-a3b
  base_url: http://localhost:11434

workflow:
  temperature: 0.1
  max_tokens: 12000
  max_repair_attempts: 5
```

The local model is required for the model-assisted stages, `pfit new`, `pfit check`, `pfit jax`, and optional semantic diagnosis. Once a session has a validated YAML configuration and generated JAX script, `pfit run` can be repeated without calling the language model.

3.1.2 Requirements on User Input

A new problem begins with the following directory structure:

```
sessions/<session-name>/
  inputs/
    user_info.txt
    <data-file>.csv
```

where the `user_info.txt` file is used to describe the problem setup to the language model. The user should try to make the file as complete as possible to make the estimation process error-free. Section 3.2 demonstrates a sample problem end to end, which shows how this input can be set up. Additionally, the repo's session dir has several solved examples to demonstrate additional set ups.

A few notes on the input: Equations may be written in ordinary mathematical notation, and short code snippets, and do not need to be syntactically correct code. Names must be used consistently: States, parameters, intermediate rates, observables, and forcing inputs should have distinct

names that are valid Python identifiers. Names that differ by capitalization or underscores are treated as different quantities. Every symbol used in an ODE or observable must be defined as a state, parameter, forcing input, or intermediate formula. If required information is missing or contradictory, `pfit-new` reports targeted `missing_inputs` and stops.

Table 1 summarizes the required information in the input.

Table 1: Information required to define a new parameter-estimation problem

Required information	**Description**
ODE system	Every integrated state must be named and have exactly one right-hand-side expression. Intermediate rates and formulas referenced by the equations must also be defined.
Dataset files and mappings	Every CSV column must be interpretable as time, observed output, forcing input, or uncertainty column. If a column is a forcing or uncertainty column, the user prompt should say so explicitly; otherwise the model may incorrectly treat it as an observed output. maybe link to example
Initial conditions	A finite initial value must be provided for every integrated state. For fitting a multi-dataset model, the user should explicitly say what the initial conditions are for each dataset.
Trainable parameters	Every fitted parameter must have a name, finite lower and upper bounds, and an explicit choice of linear or logarithmic search coordinates.
Fixed parameters	Every fixed parameter must have a name and finite numerical value. Values stated in the input specification are treated as authoritative.
Derived observables	Any simulated quantity compared with data must be defined mathematically in terms of states, parameters, and previously defined formulas.
Measured forcing inputs	Known time-dependent inputs must be associated with declared CSV columns. The current workflow supports finite forcing histories with linear interpolation.
Loss definition	The objective must be stated mathematically, including simulated-to-measured mappings, normalization, transformations, MSE or RMSE reductions, channel and experiment aggregation, weighting, penalties, and missing-data behavior. For more information see Section 3.1.4
Numerical and output requirements	Known stiffness, required integrator behavior, tolerances, and any necessary output or writeout definitions should be stated when they are known to assist the model in choosing tolerances and integrators.

Based on the problem provided, the model automatically chooses between an explicit or implicit high order integrator, makes an estimation on maximum solver steps that should be used, calibrates the per-state tolerances that should be used, and performs solver accuracy checks after the selection of these. Units must already be consistent across the equations, data, initial conditions, parameter values, bounds, and objective. The framework does not infer units or perform unit conversions.

3.1.3 Requirements on the dataset files

The framework does not modify user data files, so each CSV must already be in a form that the loader, validator, and ODE solver can use. The requirements are summarized in Table 2.

Table 2: Requirements for CSV data supplied to the pfit workflow. tables read like AI content, fix

Requirement	Remarks
Comma-separated text	Files must use ordinary comma-separated fields. Tab-delimited and other separator formats are not supported by the fitting-data loader.
Required header row	New sessions require a header row. The first header (at index 0) must be named exactly `time`.
Time is the first column	Column 0 contains the recorded time grid. Time values must be numeric, finite, and strictly increasing.
Minimum size	Each CSV must contain at least two numeric data rows and at least one declared non-time column.
Rectangular structure	Every row must contain the same number of fields, and the CSV width must match the declared column layout. A trailing comma may create an additional empty field and therefore cause a width mismatch.
Numeric data cells	Data cells must contain numeric values or permitted missing-measurement values, such as `NaN`.
Declared column meanings	Every non-time CSV column must be described in the user problem statement. The statement should say whether the column is a measured state, a measured derived observable, a measurement uncertainty, or a known forcing input. The generated YAML configuration records these meanings explicitly, and `pfit check` verifies that the generated mappings are consistent with the CSV headers and problem statement.
Measurement columns	A measurement column must map unambiguously to an integrated state or derived observable.
Custom loss masking	Custom and uncertainty-weighted losses are responsible for applying valid masks before logarithms, normalization, or division. The framework does not guarantee safe masking for arbitrary custom loss algebra.
Uncertainty columns	An uncertainty column must explicitly identify the measurement column to which it belongs. The framework can use uncertainty for a measurement as a normalizing metric when building an RMSE loss.
Forcing columns	A forcing column represents a known input history rather than an observation to be fitted. The current workflow uses linear interpolation.
Initial conditions	Initial conditions are supplied in `user_info.txt` and recorded in the generated configuration. the user should ensure that any initial condition provided should also be provided in the CSV. However, the start time does not have to be t=0.

Continued on next page

Table 2 – continued from previous page

Requirement	Remarks
Multiple experiments	All CSVs in one fitting session must use the same ordered column names, roles, measurement mappings, uncertainty relationships, and units. Their row counts, time grids, time ranges, numerical values, and declared initial conditions may differ.
Consistent units	Time, measurements, forcing inputs, initial conditions, model equations, and parameter values must already use compatible units. The framework neither infers units nor performs unit conversion.

When generated model code is executed, the time column is passed separately as the integration grid. Consequently, `dataset[:, 0]` in generated Python or JAX code refers to the first *non-time* CSV column, not to time. Users should keep this indexing convention in mind when reviewing the generated loss, forcing, and output functions.

Fitting multiple datasets simultaneously add example of file labeling for datasets: link to sneyd

Several experimental records may be fitted simultaneously when they describe the same dynamical system under different experimental conditions. The framework estimates one shared parameter vector and uses the same equations, fixed parameters, observables, loss implementation, and numerical settings for every record. Experiments that require different fitted parameter values should instead be placed in separate sessions.

Each experiment is associated with its own CSV file and may have its own time grid, number of observations, initial and final times, initial-condition overrides, and forcing history. The problem statement must identify every file and state explicitly what differs between the experiments. For example:

```
Experiments:
- run_low.csv:  A(0) = 0.1, B(0) = 0
- run_mid.csv:  A(0) = 1.0, B(0) = 0
- run_high.csv: A(0) = 10.0, B(0) = 0
```

The scalar loss is evaluated separately for each experiment using that record's own data, time grid, initial conditions, and forcing values. If there are N_{exp} experiments with scalar losses $\mathscr{L}_e$, the fitted objective is

$$\mathscr{L}_{\text{total}} = \frac{1}{N_{\text{exp}}} \sum_{e=1}^{N_{\text{exp}}} \lambda_e \mathscr{L}_e. \tag{11}$$

By default, λ_e=1. This is an equal-weight mean of complete experiment losses, not a pooled loss over every observation. The mathematical loss should therefore define a suitable reduction and normalization within each experiment. Otherwise, an experiment whose scalar loss is numerically much larger may dominate the optimization despite the equal experiment weights.

Each experiment receives its own saved trajectory:

```
result_solution_exp1.csv
result_solution_exp2.csv
...
```

The run summary records the loss for each experiment as well as the equal-experiment-mean aggregation. Run snapshots, gradient-only restarts, sloppiness analysis, and post-fit diagnosis all use the complete set of experimental records.

3.1.4 Objective function

The construction of the objective is central to the quality of the obtained estimates. The objective **must** capture all constraints on the ODE solution that the user requires. Let the model solution be written as $\mathbf{y}(t;\theta)$, where θ denotes the fitted parameters. A general user-specified constraint can be written as

$$f_i(t,\mathbf{y}(t;\theta),\theta) \leq 0, \tag{12}$$

where f_i may represent a physical bound, a monotonicity requirement, a relationship between states, or a constraint on a derived observable. Such constraints can be incorporated into the objective through weighted penalty terms. A typical objective has the form

$$\mathscr{L}(\theta) = \mathscr{L}_{\text{data}}(\theta) + \sum_{i=1}^{N_c} w_i \, \mathscr{P}_i[f_i(t,\mathbf{y}(t;\theta),\theta)], \tag{13}$$

where $\mathscr{L}_{\text{data}}$ measures agreement between the simulated outputs and the data, w_i is the weight assigned to the i-th constraint, and $\mathscr{P}_i$ is a nonnegative penalty function. The penalty function should be smooth to allow stable gradient computation.

This requires some user expertise and insight, as an underconstrained problem may allow unanticipated or undesired dynamics, while an overconstrained problem may prevent the optimizer from finding a feasible solution during the global optimization stage. The user may choose to allow the agent to formulate the loss from the problem specification. This can work for simpler problems, but for more complex problems it is advised that the user specify how the loss is constructed from the ODE solution manually. Broad best practices for constructing the objective function are listed in Table 3.

3.2 Running example

3.2.1 Starting a new session

After preparing the input statement and data files, the session directory should contain only the user-supplied files:

```
sessions/<session-name>/
  inputs/
    user_info.txt
    <data-file>.csv
    ...
```

Table 3: Best practices for building an objective function

Best Practice	Remarks
Every term of the overall objective should be of similar scale in magnitude (preferably close to unity)	It is recommended to divide each objective term by a scale belonging to its observable, like its own peak, value range, or measurement uncertainty. This enables more uniform estimation of parameters, and a relative importance of terms can be controlled by setting channel weights. Several assumptions in the framework are built around this scale being close to unity.
Weighting of samples within a trajectory	Some trajectories to fit to might only carry relevant dynamics in small subsets of the overall time history, but by default each sample in time history may be weighted equally. This can lead to poor estimates, and the loss formulation can be weighted unequally in time to mitigate this
Scaling of value *within* a loss term	In some instances, the data being estimated can span several orders of magnitude within a single channel and trajectory. Users are advised to use logarithmic scalings to allow for for uniform contributions to the objective in such cases
Differentiability of constraints	To allow gradient-based refinement, constraint penalties should be written in smooth or nearly smooth forms whenever possible. When a threshold-like behavior is needed, users should prefer a smooth approximation, such as a steep sigmoid or soft transition.

Create the session with:

```
pfit new sessions/<session-name>
```

The `pfit new` command is the first model-assisted stage of the workflow. It reads the natural-language problem statement in `user_info.txt` together with the headers and numerical contents of the CSV files. The configured local model is then asked to extract the structured fitting problem: the experiments to include, trainable and fixed parameters, integrated state variables, initial conditions, right-hand-side equations, derived observables, measured outputs, forcing inputs, uncertainty columns, and the intended loss definition.

The framework renders the extracted information into a session configuration and readable model file, then applies deterministic checks before writing the accepted artifacts. These checks verify that required fields are present, CSV headers are accounted for, parameter bounds and initial conditions are finite, state and parameter names are consistent, forcing and uncertainty columns have valid roles, and the generated model exposes the expected Python interfaces.

When successful, `pfit new` writes the initial generated artifacts:

```
sessions/<session-name>/
  inputs/
    user_input.yaml
  generated/
    user_model.py
```

```
agent_logs/
```

The file `inputs/user_input.yaml` contains the structured configuration used by later stages, including experiment definitions, data-column mappings, parameter bounds, initial conditions, solver settings, and output settings. The file `generated/user_model.py` contains a readable NumPy implementation of the ODE right-hand side, derived observables, loss calculation, and output writeout. The `generated/agent_logs/` directory stores the local-model requests and responses used during construction.

If the problem statement is incomplete or internally inconsistent, `pfit new` stops before accepting generated files and reports targeted `missing_inputs`. These messages indicate what the user should clarify in the input statement or data files, such as missing parameter bounds, ambiguous CSV column meanings, absent initial conditions, or an undefined observable. After updating the user-supplied inputs, the session can be regenerated with:

```
pfit new sessions/<session-name> --overwrite
```

Two state drug response model The tutorial uses a two-state drug-response model fitted simultaneously to pulse and washout experiments. Both CSV files use the same columns,

```
time,dose_rate,signal
```

but have different forcing histories and initial conditions. The corresponding `user_info.txt` is:

```
Problem:
Fit pulse.csv and washout.csv using one shared parameter set.

Experiments and initial conditions:
pulse.csv:   D(0) = 0, R(0) = 0
washout.csv: D(0) = 0, R(0) = 0.3

CSV columns:
time: time in minutes
dose_rate: known forcing input in concentration per minute;
           use linear interpolation
signal: measured value of signal_model in arbitrary units

Trainable parameters:
k_elim:   bounds [0.01, 10], logarithmic
k_on:     bounds [0.001, 10], logarithmic
k_off:    bounds [0.001, 10], logarithmic
baseline: bounds [50, 150], linear
gain:     bounds [1, 200], linear
```

```
Fixed parameters:
None.

ODE system:
dD/dt = dose_rate - k_elim*D
dR/dt = k_on*D*(1 - R) - k_off*R

Observable:
signal_model = baseline + gain*R
CSV column signal measures signal_model.

Loss:
For each experiment e, use only finite signal measurements.

scale_e = max(abs(finite signal values in experiment e))

L_e = sqrt(mean(
    ((signal_model_e - signal_e)/scale_e)^2
))

The total loss is the equal-weight mean:
L = (L_pulse + L_washout)/2.

Outputs:
Write time, D, R, measured signal, and signal_model.
```

This statement supplies the scientific decisions required by the workflow. The local model is not expected to choose missing equations, parameter bounds, initial conditions, data mappings, units, or loss terms. The measured CSV column `signal` is valid here because the statement explicitly maps it to the derived observable `signal_model`. In general, measured columns should map unambiguously to declared integrated states or derived observables, and forcing or uncertainty columns should be identified by role.

3.2.2 Checking the session

read again After reviewing the files produced by `pfit new`, check the session with:

```
pfit check sessions/<session-name>
```

This command reads the generated YAML, source model, and every declared CSV file. The framework first performs deterministic checks by parsing these files and comparing their declarations. It verifies that the data dimensions and headers match the configured columns, parameter values and bounds are valid, initial conditions are complete, model names are consistent, and the source model uses the expected interfaces and column indices.

The loss is also compared with the generated implementation. For example, if `user_info.txt` requests an RMSE, logarithmic residuals, uncertainty weighting, finite-value masking, a particular

normalization, or a specific experiment aggregation, the generated loss must contain the corresponding operation. However, if an explicit user-supplied loss is provided, it overrules any automated loss function generation. The check surfaces potential scaling issues as warnings, which are crucial to fit quality.

The deterministic checks also look for several common failure modes: non-smooth or branch-like dynamics that may be difficult to differentiate, measured columns with large dynamic range, and loss implementations that appear to violate any explicit loss instructions in `user_info.txt`.

For the running example, the framework verifies that both CSV files use the same ordered columns; that `dose_rate` is declared as a finite forcing input; that `signal` maps to the derived observable `signal_model`; and that the two experiments retain their specified initial conditions. It also checks that the generated loss uses finite signal measurements, applies the stated per-experiment normalization, calculates an RMSE for each experiment, and preserves the equal-experiment aggregation.

The report is written to:

`sessions/<session-name>/generated/user_input_check.txt`

After performing a check, three categories of errors are surfaced. Critical errors must be corrected before continuing. Warnings identify limitations or potentially fragile assumptions, while recommendations identify choices that the user may wish to reconsider. By default, the configured local model may also review the session description, generated source, and deterministic evidence for semantic inconsistencies.

It should be noted that passing the checks confirms that the session satisfies the implemented structural, numerical, and consistency rules for that stage. It does not establish scientific correctness or model optimality for the data.

3.2.3 Generating the JAX model

Once the session has passed the checks described above, generate the executable JAX model by running:

`pfit jax sessions/<session-name>`

This command converts the readable NumPy model in `generated/user_model.py` into the differentiable JAX implementation used for parameter estimation. The workflow breaks the code generation into smaller steps and completes translation. The resulting script is written to:

`sessions/<session-name>/generated/generated_script.py`

A simplified excerpt of the transformed JAX system for this example is:

```
@jax.jit
def user_defined_system(t, y, other_args):
    constants = other_args["constants"]
    trainable_variables = other_args["trainable_variables"]
    dataset = constants["dataset"]
```

```
    t_eval = constants["t_eval"]

    k_elim = unscale_value(trainable_variables[0], 0.01, 10.0, True)
    k_on = unscale_value(trainable_variables[1], 0.001, 10.0, True)
    k_off = unscale_value(trainable_variables[2], 0.001, 10.0, True)
    baseline = unscale_value(trainable_variables[3], 50.0, 150.0, False)
    gain = unscale_value(trainable_variables[4], 1.0, 200.0, False)

    D = y[0]
    R = y[1]
    dose_rate = jnp.interp(t, t_eval, dataset[:, 0])

    dDdt = dose_rate - k_elim * D
    dRdt = k_on * D * (1.0 - R) - k_off * R
    return jnp.array([dDdt, dRdt])

@jax.jit
def _compute_loss_value(constants, trainable_variables,
                        solution_time, solution):
    dataset = constants["dataset"]
    baseline = unscale_value(trainable_variables[3], 50.0, 150.0, False)
    gain = unscale_value(trainable_variables[4], 1.0, 200.0, False)

    R = solution[:, 1]
    signal_model = baseline + gain * R
    signal = dataset[:, 1]
    mask = jnp.isfinite(signal)
    scale = jnp.max(jnp.abs(signal[mask]))
    residual = (signal_model[mask] - signal[mask]) / scale
    return jnp.sqrt(jnp.mean(residual**2))
```

This excerpt omits the surrounding scaffolding, but it shows the main transformation: parameters are mapped from optimizer coordinates to physical values, the forcing input is interpolated from the non-time dataset columns, and the functions are written in a `jit-compilable` method.

After generation, the framework first checks that the script provides the required runtime interface for integration, loss evaluation, and output writing. It then imports the script, runs the model for each dataset, verifies that trajectories and losses are finite, and evaluates the generated code on some sampled parameter points. The JAX implementation is also compared with the readable NumPy source model at sampled states and parameter values. The comparison checks differential-equation values, observable predictions, output columns, and losses where applicable.

Automated setup tuning The framework adaptively tunes some heuristics of the ODE solver at this stage. For instance, if too few sampled parameter points integrate successfully because the solver reaches its step limit, the workflow may increase `max_steps` up to the configured recovery ceiling and rerun the same validation. If recovery succeeds, the accepted step budget is written back to the YAML configuration. Similarly, proposed state-scaled tolerances or population-stage

relaxations are accepted only when they preserve finite losses on a meaningful fraction of sampled candidates. Solver-accuracy checks then compare selected feasible probes with tighter reference solves, until the tolerance gets too tight for successful integration.

Repair of incorrect translation If a translation comparison or interface check fails, the local model does a targeted repair on the fragment responsible for the failure. The number of repair attempts is limited. Before fitting, the user should inspect `generated_script.py` and the generated reports and confirm that the equations, parameter names, observable definitions, loss function, output columns etc. match the intended workflow

Finally, re-verify that the complete session is ready to run:

```
pfit check sessions/<session-name> --ready
```

3.2.4 Running the fit

After the JAX model has been generated and the session passes readiness checks, run parameter estimation with:

```
pfit run sessions/<session-name>
```

This stage does not require the local LLM model, and can be run repeatedly on a CPU-only device. It uses the accepted YAML configuration, CSV files, and `generated_script.py`. Before fitting starts, the same deterministic readiness checks are repeated to ensure that the generated script is current.

Each invocation creates a new preserved run directory under:

```
sessions/<session-name>/outputs/
```

The default run performs population-based global exploration followed by gradient-based refinement. The population stage searches the bounded parameter space in the normalized coordinates defined by the YAML configuration. The gradient stage starts from the best population result and refines it using the differentiable JAX loss.

For an additional refinement starting from a previous run, use:

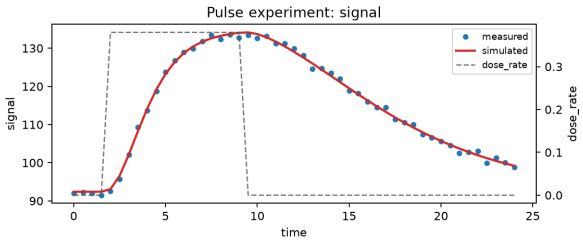
```
pfit run sessions/<session-name> gradient-only --from-run <run-id>
```

By default, the run also performs post-fit sloppiness analysis at the final parameter estimate. A completed run indicates that the configured optimization procedure finished and wrote its artifacts.

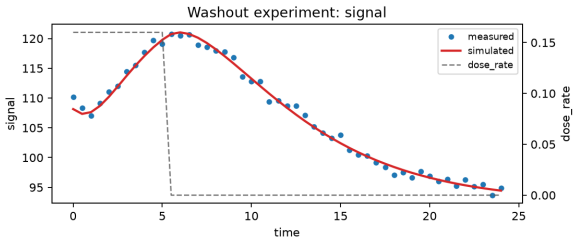
3.2.5 Diagnosing a completed fit

After parameter estimation has finished, examine the result with:

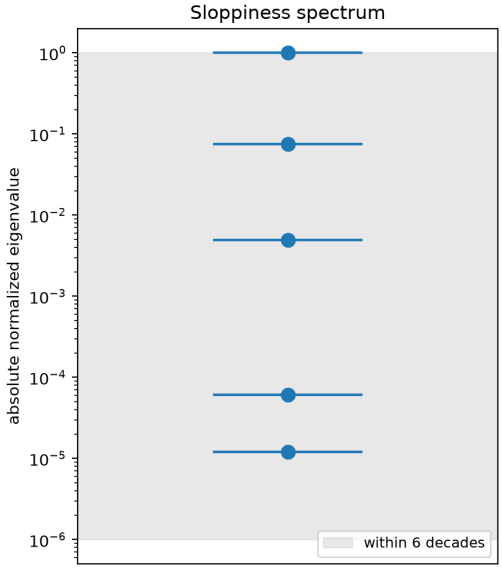
```
pfit diagnose sessions/<session-name> <run-id>
```



(a) Pulse experiment.



(b) Washout experiment.



(c) Post-fit sloppiness spectrum.

Figure 1: Example diagnostic outputs for the pulse/washout running example.add more detail

The run identifier is the name of a directory under `sessions/<session-name>/outputs/`. If it is omitted, the most recent completed run is selected. For each dataset, the command recreates the fitted trajectory and compares it with the measurements. In the running example, separate plots are produced for the pulse and washout experiments, as seen in Figure 1. These show the measured signal together with the fitted `signal_model`.

The diagnosis reports residual summaries, including root-mean-square error, mean absolute error, bias, largest absolute residual, and time-pattern evidence. The report also checks whether fitted parameters are close to their bounds, whether optimizer logs suggest that improvement was still occurring, and whether the run used a full fit or a gradient-only restart. If post-fit sloppiness analysis was performed during `pfit run`, the diagnosis includes its status and summary, and a plot of the sloppiness diagnosis as visible in Figure 1c.

A representative excerpt from the file `fit_diagnosis.txt` for this example is:

```
Fit diagnosis

Artifacts:
```

```
- final_design_point.csv: present
- result_solution_exp1.csv: present
- result_solution_exp2.csv: present
- de_fitting.log: present
- NODE_fitting.log: present
- fitting_error.txt: missing

Optimizer summary:
- DE: 120 iterations, first loss 8.0936E-02,
      best loss 5.7755E-03, final loss 5.7755E-03
- NODE: 21 iterations, first loss 5.7754E-03,
        best loss 5.7753E-03, final loss 5.7753E-03

Experiments: 2
Aggregation: equal_experiment_mean
- Experiment 1: pulse.csv; loss=0.005219267638936577;
  output=result_solution_exp1.csv
- Experiment 2: washout.csv; loss=0.006331386422963499;
  output=result_solution_exp2.csv

Scientific diagnosis:
- Status: ok
- Refinement stopped at its iteration budget.
- [BUDGET] Refinement stopped at its iteration budget.
  Next: Assess the final gradient and trajectory plots before
  deciding whether a gradient-only restart is worthwhile.
- Figure: fit_exp1.png
- Figure: fit_exp2.png
- Detailed evidence: scientific_diagnosis.json
```

When sloppiness analysis is enabled, a representative `sloppiness_report.txt` excerpt is:

```
Post-fit sloppiness
Status: ok
Method: finite-difference
Coordinates: log10(k_elim), log10(k_on), log10(k_off), baseline, gain
Verdict: no weak curvature directions at the specified relative floor
Weak modes: 0
Negative modes: 0
Normalized gradient infinity norm: 5.63383e-06
Eigenvalues (largest to smallest):
  mode 1: 1.68783850e+01
  mode 2: 1.26587665e+00
  mode 3: 8.25273064e-02
  mode 4: 1.03427087e-03
  mode 5: 2.01732936e-04
```

Table 4: Memory requirements, average completion time (all 5 stages, passed cases only) and completion rates for attempted ODE estimation problems

Model	Memory Req. (GB)	Mean completion time (s)	Pass	Fail	Success Rate
Qwen2.5-Coder 32B (A)	19.9	110.0	15	8	71.4%
Qwen2.5-Coder 14B (B)	9.0	93.8	13	10	61.9%
Qwen3.8-27B (C)	17.7	235.6	19	4	90.5%

```
Positive spectrum span: 4.92255 decades
STIFFEST direction: k_off(-0.64), k_elim(-0.63),
                    k_on(+0.43), baseline(+0.02)
SLOPPIEST direction: gain(+1.00), baseline(-0.08),
                     k_on(-0.02), k_elim(+0.01)
Warning: Local curvature only: not proof of structural/global
identifiability or convergence.
```

By default, the framework may ask the configured model to summarize the computed evidence and suggest next steps. This returns a semantic interpretation of the results. To produce only the deterministic report without using the local model, which may be desired if GPU access is only transient, use:

```
pfit diagnose sessions/<session-name> <run-id> --deterministic-only
```

4 Model Performance

add links to full fits on repo

This section discusses the tradeoffs of using different LLMs for the process.The workflow is evaluated on a suite of ODE parameter-estimation tasks using three locally served Ollama models. The focus is on problems with experimental data, and several examples from biological sciences are taken directly from an external, community-curated benchmark collection (the PEtab benchmark suite [18]).

The framework is evaluated using three different LLM models, with the comparison run on an NVIDIA H100 80GB HBM3 GPU. The Ollama request configuration used a 32k token context window, temperature 0.1, and a 12k token generation budget. The temperature value, which controls randomness, is kept deliberately low for this application which requires high reproducibility. Table 4 discusses memory approximate memory consumed for each model, along with their success rates on tested problems. The tests show that using newer and larger models has a higher success rate, but has larger hardware requirements. For example, Qwen2.5-Coder 14B can also be run for these problems on an Apple M4.

Table 5 summarizes the observed completion behavior, where identical tasks were provided to 3 different models. Qwen3.8-27B completed 19 of 23 tasks (90.5%), compared with 15 of 23 for the 32B baseline (71.4%) and 13 of 23 for the 14B baseline (61.9%). The Qwen3.8 model was slower in wall-clock time, partly because it is a thinking model and its reasoning tokens are included in the generation budget, but it was substantially more reliable on the larger biochemical

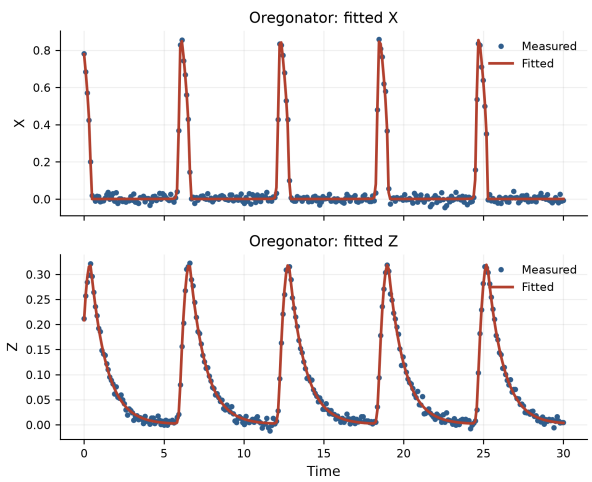
and multi-experiment cases. The Qwen2.5-Coder 14B model was fast and often adequate for small canonical systems, but it failed on several larger multi-experiment or biochemical prompts during `pfit new`.

The retained examples span a deliberately broad range of ODE fitting tasks. They include low-dimensional textbook dynamics, stiff chemical kinetics, oscillatory systems, biochemical signaling networks, pharmacokinetics, hysteresis, measured forcing functions, derived observables, and multi-experiment shared-parameter fits. The diversity of examples demonstrates that the workflow works well in qualitatively different modeling regimes.

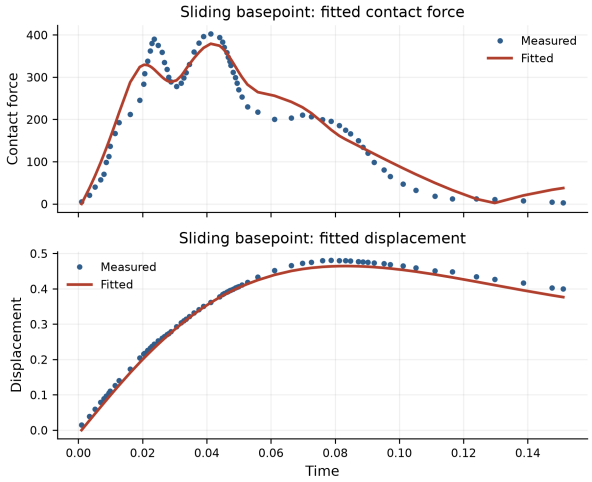
Table 5: End-to-end local workflow completion by models across a diversity of ODE tasks. A check mark denotes successful completion of all five pfit stages, but does not include RMSE as a benchmark, since that depends on suitability of model dynamics and noise levels of data. Models A,B,C are as described in Table 4

Case	Model A	Model B	Model C	Source or model family
ARC fitting	✓	✓	✓	Accelerating-rate calorimetry battery example [19]
Boehm STAT5	✓	✓	✓	STAT5 signaling model [20]
Lotka–Volterra	✓	✓	✓	Predator–prey equations [21, 22]
MAPK cascade	✓	✓	✓	Kholodenko MAPK cascade [23]
Oregonator	✓	✓	✓	Field–Noyes/Tyson BZ reaction model [24, 25]
Piezo Bouc–Wen	✓	✗	✓	Bouc–Wen hysteresis family [26, 27]
Robertson	✓	✓	✓	Stiff chemical kinetics model [28]
Sliding basepoint	✗	✓	✓	Frictional sliding-basepoint example [29]
Theophylline	✓	✓	✓	One-compartment pharmacokinetic example [30]
Van der Pol	✓	✓	✓	Nonlinear oscillator [31]
Cascaded tanks	✓	✗	✓	Two-tank process-control example [32]
Multi-exponential decay	✓	✓	✓	Synthetic multi-experiment decay example [33]
Sneyd IP_3 receptor	✗	✓	✓	IP_3 receptor model [34]
Beer indigoidine	✓	✓	✓	Indigoidine production dataset [35]
Raia IL-13	✓	✗	✓	IL-13/JAK2/STAT5 lymphoma model [36]

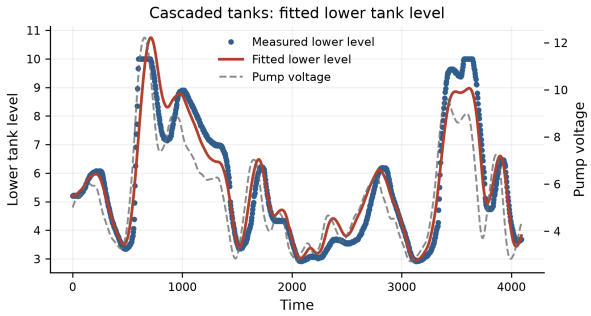
A few representative completed fits are shown in Figure 2. The Oregonator example tests a stiff oscillatory chemical system with only a subset of states observed; the sliding-basepoint case tests a



(a) Oregonator oscillatory chemistry.



(b) Sliding-basepoint friction model.



(c) Cascaded tanks with measured forcing.

Figure 2: Representative measured-versus-fitted outputs from the retained session examples.add more detail and write para text

nonsmooth frictional model with derived force and displacement observables; the cascaded-tanks example tests a measured forcing input.

The completed examples are included in the repository under `sessions/` at the repo. Each curated session contains the user prompt and data under `inputs/`, the generated model artifacts under `generated/`, and at least one completed fitting run under `outputs/`. The `outputs/` directory for each retained case also contains one or more `fit_exp*.png` figures showing the fitted trajectory against the data. These sessions are intended to serve both as examples for users and as reproducibility evidence for the local-agent workflow.

It should be noted that successful execution of the workflow is not the same thing as proving that the proposed ODE is a perfect description of the system. In several cases the framework produces a stable, finite, and visually useful fit while the residual structure still shows systematic mismatch. This can occur due to external factors such as the model being an imperfect representation of system dynamics, measurement noise and more.

5 Summary and Conclusions

A note on model appropriateness

The framework finds a good solution based on equations, data, losses and constraints describe the problem. One of the main purposs of the framework is to allow users to easily fit to noisy, real world data. In most applications, this means that the model is an approximate representation of real world dynamics, and the framework cannot guarantee perfect fits.

Competing Interests

The author has no conflicts of interest to declare that are relevant to the content of this chapter.

References

References

[1] Andreas Raue, Clemens Kreutz, Thomas Maiwald, Julie Bachmann, Marcel Schilling, Ursula Klingmuller, and Jens Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15): 1923–1929, 2009. doi: 10.1093/bioinformatics/btp358.

[2] Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS Computational Biology*, 3(10):e189, 2007. doi: 10.1371/journal.pcbi.0030189.

[3] Keyan Miao, Liqun Zhao, Han Wang, Konstantinos Gatsis, and Antonis Papachristodoulou. Opt-odenet: Neural ode controller design with differentiable optimization layers for safety and stability. In Necmiye Ozay, Laura Balzano, Dimitra Panagou, and Alessandro Abate, editors, *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*, volume 283 of *Proceedings of Machine Learning Research*, pages 1217–1229. PMLR, 04–06 Jun 2025. URL `https://proceedings.mlr.press/v283/miao25a.html`.

[4] Johannes Heeg, Yunlong Song, and Davide Scaramuzza. Learning quadrotor control from visual features using differentiable simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4033–4039, 2025. doi: 10.1109/ICRA55743.2025.11128641.

[5] Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579, 2024.

[6] Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *Advances in Neural Information Processing Systems*, 38:59765–59789, 2026.

[7] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam: an llm-based multi-agent framework for cfd. *arXiv preprint arXiv:2407.21320*, 2024.

[8] Ling Yue, Peijing Xu, Tingwen Zhang, Nithin Somasekharan, and Shaowu Pan. Foamgpt: Fine-tuning large language model for agentic automation of cfd simulations with openfoam. *Advances in Neural Information Processing Systems*, 2025.

[9] Zhaoyue Xu, Long Wang, Chunyu Wang, Yixin Chen, Qingyong Luo, Hua-Dong Yao, Shizhao Wang, and Guowei He. Cfdagent: A language-guided, zero-shot multi-agent system for complex flow simulation. *Physics of Fluids*, 37(11), 2025.

[10] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam 2.0: Large language model driven chain of thought for automating cfd simulation and post-processing. *arXiv preprint arXiv:2502.00498*, 2025.

[11] Donggeun Park, Hyeonbin Moon, and Seunghwa Ryu. A self-correcting multi-agent llm framework for language-based physics simulation and explanation. *npj Artificial Intelligence*, 2, 2026. doi: 10.1038/s44387-025-00057-z.

[12] Guillaume Damblin, Mathieu Couplet, and Bertrand Iooss. Numerical studies of space filling designs: Optimization of latin hypercube samples and subprojection properties. *Journal of Simulation*, 7(4):276–289, 2013.

[13] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.

[14] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.

[15] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.

[16] Yingbo Ma, Vaibhav Dixit, Michael J Innes, Xingjian Guo, and Chris Rackauckas. A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions. In *2021 IEEE high performance extreme computing conference (HPEC)*, pages 1–9. IEEE, 2021.

[17] Ryan N Gutenkunst, Joshua J Waterfall, Fergal P Casey, Kevin S Brown, Christopher R Myers, and James P Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS computational biology*, 3(10):e189, 2007.

[18] Helge Hass, Carolin Loos, Elba Raimúndez-Álvarez, Jens Timmer, Jan Hasenauer, and Clemens Kreutz. Benchmark problems for dynamic modeling of intracellular processes. *Bioinformatics*, 35(17):3073–3082, 2019.

[19] M. N. Richard and J. R. Dahn. Predicting electrical and thermal abuse behaviours of practical lithium-ion cells from accelerating rate calorimeter studies on small samples in electrolyte. *Journal of Power Sources*, 79:135–142, 1999.

[20] M. E. Boehm et al. Stat5a/stat5b signaling model. *Reference metadata to verify*, 2014. Placeholder entry: verify exact title, author list, journal, and pages before submission.

[21] Alfred J. Lotka. *Elements of Physical Biology*. Williams and Wilkins, 1925.

[22] Vito Volterra. Fluctuations in the abundance of a species considered mathematically. *Nature*, 118:558–560, 1926.

[23] Boris N. Kholodenko. Negative feedback and ultrasensitivity can bring about oscillations in the mitogen-activated protein kinase cascades. *European Journal of Biochemistry*, 267: 1583–1588, 2000.

[24] Richard J. Field and Richard M. Noyes. Oscillations in chemical systems. iv. limit cycle behavior in a model of a real chemical reaction. *The Journal of Chemical Physics*, 60:1877–1884, 1974.

[25] John J. Tyson. A quantitative account of oscillations, bistability, and traveling waves in the belousov-zhabotinskii reaction. *Annals of the New York Academy of Sciences*, 452:109–122, 1985.

[26] Robert Bouc. Modèle mathématique d'hystérésis. *Acustica*, 24:16–25, 1971.

[27] Y. K. Wen. Method for random vibration of hysteretic systems. *Journal of the Engineering Mechanics Division*, 102:249–263, 1976.

[28] H. H. Robertson. The solution of a set of reaction rate equations. *Numerical Analysis: An Introduction*, pages 178–182, 1966.

[29] Jean Jacques Moreau. Unilateral contact and dry friction in finite freedom dynamics. In *Nonsmooth Mechanics and Applications*, pages 1–82. Springer, 1988.

[30] José C. Pinheiro and Douglas M. Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, 2000.

[31] Balthasar van der Pol. On relaxation-oscillations. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926.

[32] Maarten Schoukens, Per Mattsson, Torbjörn Wigren, and Jean-Philippe Noël. Cascaded tanks benchmark combining soft and hard nonlinearities. In *Workshop on Nonlinear System Identification Benchmarks*, Brussels, Belgium, 2016.

[33] Gaspard Riche de Prony. Essai expérimental et analytique sur les lois de la dilatabilité de fluides élastiques et sur celles de la force expansive de la vapeur de l'eau et de la vapeur de l'alkool, à différentes températures. *Journal de l'École Polytechnique*, 1:24–76, 1795.

[34] James Sneyd and Jean-François Dufour. A dynamic model of the type-2 inositol trisphosphate receptor. *Proceedings of the National Academy of Sciences*, 99:2398–2403, 2002.

[35] M. Beer et al. Indigoidine production model. *Molecular BioSystems*, 2014. Placeholder entry: verify exact title, author list, and pages before submission.

[36] V. Raia et al. Dynamic mathematical modeling of il-13-induced signaling in hodgkin and primary mediastinal b-cell lymphoma. *Cancer Research*, 2011. Placeholder entry: verify exact title, author list, volume, and pages before submission.

Tables

Table 6: Suggested input files for the agentic parameter estimation workflow.

File	Purpose
`problem.xml`	Structured definition of states, parameters, data, and settings
`model.py`	User-editable model and observation functions
`data.csv`	Observation data and time points
`config.yaml`	Solver, optimizer, output, and reproducibility settings

Figure Legends

Fig. 1. End-to-end agentic parameter estimation workflow. The user supplies a structured problem definition and a Python model skeleton. The agentic workflow validates the inputs, generates a differentiable calibration pipeline, runs global exploration and gradient-based refinement, and exports fitted parameters, diagnostics, and reproducible code.

Fig. 2. Example structure of the problem specification file. The schema records state variables, parameters, bounds, initial conditions, observation mappings, data paths, solver settings, and optimizer settings.

Fig. 3. Two-stage optimization strategy. A global search explores the bounded parameter space and identifies candidate regions, after which gradient-based refinement improves the best candidates using differentiable model evaluations.

Fig. 4. Example diagnostic outputs from a fitted ODE model. Suggested panels include model trajectories against observed data, residuals over time, fitted parameter values relative to bounds, and convergence history.